

MBSO - LEL 8

HiFive Unmatched CUVB → RISC-V development board.
It was designed as high performance RISC-V desktop / development system.

1 KiB = 1024 B

33 KiB = 33 × 1024 B

U74 Core RISC-V:

→ ISA:

RV64GC → G: General, C: compressed Instr.

G tells:

Integer Instr, Mul/div, Atomic Instr, Single-Precision FP, Double precision FP.

→ modes:

- 1 Machine Mode: Highest Privilege mode. Used for Hardware.
- 2 Supervisor Mode: Used by OS Kernel.
- 3 User Mode: Shell programs, user processes, cannot handle hardware.

→ L1 Instr. Cache

Stores recently used CPU Instr. for faster access.

In SiFive U74 Core, each core has L1 Instr. cache & L1 Data Cache 32kB each.

Cache Hit: Instr. already in cache.

Cache Miss: Instr. not in cache.

Core1

L1 Cache

L1 Data

Core2

L1 Cache

L1 Data

Why separate Data & Instr. Cache?

Uses Harvard Style cache design helps in simultaneous Instr. & data fetch

Cache lines: (64B data for each line) $\frac{64}{4} = 16$ Instr. are fetched
multiple Instr. are fetched due to cache lines

4-Way Set Associative:

Cache is divided into sets. Each cache contains 4 lines (ways)

Set0 → [way0][way1][way2][way3]

Set1 → [way0][way1][way2][way3]

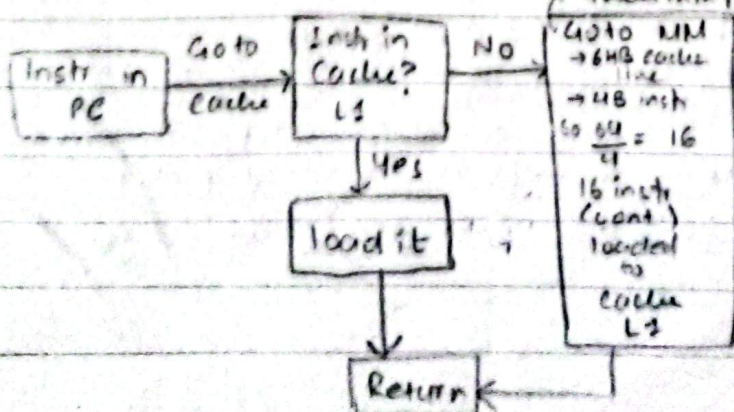
one line = 16 Instr. 1st way

one set has $16 \times 4 = 64$ Instr.

on cache hit, one line is removed (one way, a mem block)

→ 4 ways reduces cache conflicts as if 4 Instr. mapped to same set, 4 Instr. spots available. Hence, lower cache miss & low Instr. overwrite

go to L2 so every way has a diff mem. block



4 compressed Instr.

so $\frac{64}{2} = 32$ Instr. loaded

KiB = 1024B (mem unit)

KB = 1000B

∴ computer use power of 2.

$$\frac{768}{32} = 24$$

$$MIB = 1024 \times 1024$$

(1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14) (15) (16) (17) (18) (19) (20) (21) (22) (23) (24) (25) (26) (27) (28) (29) (30) (31) (32) (33) (34) (35) (36) (37) (38) (39) (40) (41) (42) (43) (44) (45) (46) (47) (48) (49) (50) (51) (52) (53) (54) (55) (56) (57) (58) (59) (60) (61) (62) (63) (64) (65) (66) (67) (68) (69) (70) (71) (72) (73) (74) (75) (76) (77) (78) (79) (80) (81) (82) (83) (84) (85) (86) (87) (88) (89) (90) (91) (92) (93) (94) (95) (96) (97) (98) (99) (100)

→ L2 Cache

→ larger but slower cache

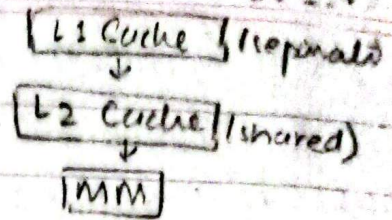
→ shared L2 cache for all cores.

→ L1 Miss → Go to L2 → L2 Miss → Go to RAM

→ it helps in multicore communication.

→ shared data access.

→ Reducing duplicate RAM fetches



L1: desk L2: bookshelf RAM: library

→ share book in h. & data.

→ When a thing is fetched from MM, it is first placed in L2 then L1.

→ 2 MiB = $2 \times 1024 \times 1024 \Rightarrow 2097152 \text{ B}$: Cache Size

→ Each set of cache has 16 available slots (16 way)

→ 4 Banks

4 separate room in library. Multiple doors to search.

→ parallel access for multicore.

→ They are like access port.

Request 0 → Bank 0 (0-511 set)

Request 1 → Bank 1 (512-1023 set)

Request 2 → Bank 2 (1024-1535 set)

Request 3 → Bank 3 (1536-2047 set)

$$\text{Total line} = \frac{2097152}{64} = 32768 \text{ lines}$$

$$\text{Set} = \frac{32768}{16} = 2048 \text{ sets}$$

$$\text{lines/way} = 2048$$

$$\text{Set per bank} = \frac{2048}{4} = 512$$

→ ECC Support:

Error Correcting Code, hardware feature that helps detect & fix memory errors automatically.

→ RAM can sometimes flip bits due to noise, hardware fault, etc so we need ECC for detection & correction

For Every Data placing: Data + ECC bits → stored in Mem.

When Reading: Data + ECC bit → check → error detect & correct

→ Physical Memory Protection: (which mem region can R, W, Exec)

→ Actual addresses in RAM hardware. (Data also includes mapping area. with its own starting, ending point & perhaps we can configure these 3 regions individually like R=Yes, W=Yes Exec=No)

→ 4 KiB → 4096 B each block (1 page)

so $4096 \times 8 = 32768 \text{ Bytes}$

→ Memory Management Unit → hardware block in CPU that translates virtual addresses into physical addresses & enforces memory protection

→ Prog. in Virtual Addr. But MM uses physical Addr.

474 - RISC-V PROCESSOR PIPELINE

- InOrder Execution : Sequential.
- Out Issue : Executes 2 instr. per cycle.
- Stages/Pipeline:

IF: CPU reads instr. from L1 cache & sends instr. to next stage

LS ALU : A

(F1: PC req. instr. from L1 cache) can be comp

F2: Instr. is received / aligned & prepared.

B M D FP ALU : B

D1: decode instr.

D2: Register file read + control signals prepared

Ag: Calculating memory addr.

M1: L1 lookup

M2: Data received from L1/L2/Memory, forwarding paths activate & load value become ready

WB: Register Write

one cycle res. latency:

add x1, x2, x3

cycle = 1 = complete

cycle = 2 = x1 gets res

- Arithmetic Instr. completes in AG or M2 not in M1;
- AG: Data ready M1: No addr even if data ready
- M2: Addr for calculation.

→ Load use can happen when.

cycle	1	2	3	4	5	6	7	8	9	10	11
I1 lw	F1	F2	D1	D2	AG	M1	M2	WB			
I2 add		F1	F2	D1	D2	O	O	AG	M1	M2	WB

Iw x1, 0(x2), add x3, x1, x4 O → stall

- The latency of div: (Hence, slower)
- Div is an iterative operation. It at max shift, subtract, compare, correct quotient

→ The results are blocked until complete

→ The dependent instructions will have a stall while the independent will execute

→ Latency : 6 cycles = or 68 cycles

$$\text{Latency} = 2 \text{ cycles} + \log_2(\text{dividend}) - \log_2(\text{divisor})$$

+ 1 cycle if input is signed + 1 cycle if output is negative.

→ F2 alignment penalty:

When a word is not aligned multiple of 4, alignment takes place in F2 stage. This can happen due to compressed instructions.

Memory Blocks

4B req. by inst	0x1000 (1B)	0x1001 (1B)	0x1002 "	0x1003 "
inst	store loc storesB [x]			

Since 4B block { 0x1000, 0x1001, 0x1002, 0x1003 } words A

4B block { 0x1004, 0x1005, 0x1006, 0x1007 } words B

So, one penalty to go to next memory block & fetch word B.

→ 4 compressed instr. can be fetched per cycle (Advantage)
 since, fetching unit can fetch data Bytes instead of instr.
 8 Bytes are fetched in each cycle. So when there are
 compressed instr. $\frac{8}{2} = 4$ instrs. are fetched.
 But, that does not mean that 4 of them get issued.
 They need dual instr. only.
 fetched 1, 1, 1, 1

N cycles: 1, 1, 1, 1 N+1 cycle: 1, 1, 1, 1

Fetch unit works on demand, means, when there are no 8 Bytes,
 the instr. are fetched. So, when 4 instr. are fetched (c)
 the fetch unit remains idle.

→ Half Word Address Alignment:
 Instr. can be 2B or 4B instr. Like 0x1003 cause trap
 (exception) valid addr = 00, 02, 04, ...
 instr. can be 2B or 4B

LECTURE #9

→ Branch Prediction:

→ BTB: (16 entries) Indirect, Direct

cache that stores branch target address for faster access
 of taken branches.

F1 → BTB lookup → Is branch? → F2

(No separate cycle tho. Where lookup)

check done with F1 in parallel

Target is 8B aligned → 2 instr. are fetched for two pipelines

→ Direct Jump, miss BTB, penalty of one bubble.

Consider, 0x1000 → 0x3000, 0x1000 in PC, checks
 BTB, no target addr (a miss) so, next instr. (0x1004) is
 fetched sequentially until at decode, we know
 that the next addr is 0x3000, so a bubble is needed.

→ BHT: 3.6 KiB (Conditional Branches)

Branch taken or not prediction.

It sees that if this branch instr. is usually taken what
 then it will be taken now too.

Indirect → addr. in reg

→ ISTP: (8 entries)

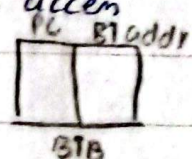
Indirect Jump: Target stored in register

Predicts the target address of indirect jump.

Eg: Jalr x0, x1, 6

→ RAS (6 entries)

Predicts correct return address of function call.
 used for: Jalr, jal, ret



BIB, THIS PC $\rightarrow$ What taught

ITIP, RAS $\rightarrow$ rely on prog. behaviour, func. call

Semantic predictors (based on instr.)
↑
ITIP

$\rightarrow$ Fetch bubble due to ITIP or RAS.

ITIP & RAS overwrite BIB, when conflict. This overwriting takes one cycle.

1024 2-bit predictors

16 BIT Table

$\rightarrow$ Mispredicted Branch Penalty:

$\rightarrow$ Four cycle Penalty:

If the branch is resolved in A4: F1 $\rightarrow$ F2 $\rightarrow$ D1 $\rightarrow$ D2 $\rightarrow$ A4.

So previous four cycles needs to be flushed & have to start from

1 cycle = Direct Jump

$\rightarrow$ Six Cycles Penalty: (Indirect Jump) (A4) 4 cycles = Indirect J

Branch resolution in M2.

F1 $\rightarrow$ F2 $\rightarrow$ D1 $\rightarrow$ D2 $\rightarrow$ A4 $\rightarrow$ M1 $\rightarrow$ M2.

[— Flush —]

eg Indirect Jump.

(M2) 6 cycles = Indirect J

$\rightarrow$ Branch Target Buffer:

$\rightarrow$ Global History Register:

Used for if (x > 0): correlating branches.
if (y > 0):

Aliasing:

Branch A: 0x1000

Branch B: 0x2000

Both mapping to same

index entry:

aliasing so

we introduced

XOR

Let GHR be 0b 10110010 Next is taken so it is like: $\leftarrow$ 01100101

It helps deciding future by looking at past like if x was taken then y may also be taken.

GHR (XOR) PC = Index of Branch History Table.

LEC 10

$\rightarrow$ L2 as cache or Direct Addressed Memory:

Cache Mode:

Direct Address Mode:

software managed memory: no tags, no replacement, policy, no cache lookup, normal load/store

CPU directly reads/writes to L2 SRAM. address register

$\rightarrow$ Loosely Integrated Memory:

Extra L2-LIM apart from the chip connected via bus

$\rightarrow$ Way Enable Register:

Used to turn on/off cache. ~~line~~ ways L2 controller controls mode of L2

L1 $\rightarrow$ Parity checking

L2 $\rightarrow$ ECC support

L2-LIM

we can disable some ways

way 0 = cache

ways 1, 2, 3 = LIM

software can directly read/write in L2

L2-LIM
(memory mode)
software access
it using mem
addr. directly

L2 Cache

Higher
LIM

After Reset

way 0 $\rightarrow$ cache

way 1 $\rightarrow$ disable/LIM

way 2

way 3

ways 1, 2, 3
 $\rightarrow$ lower LIM address

(2) Deterministic $\rightarrow$ always a hit (hence fast)

$$2^r \geq n+1$$

to find r when n given

$$2^1 \leq 2^r \leq 2^2 \text{ for 4 bit binary}$$

L2CCH

L2 Zero Device

L2 may behave as cache, direct memory & also zero device.
 → region is physical Addr. map backed by L2 SRAM. accessed without cache behavior.

1. specific address range of L2)

zero means: that 0 cache overhead. ∴ direct map

→ Error Correcting Code:

→ Hamming Distance:

Number of bit positions in which two binary codes differ

L2 Zero Device → L2 SRAM used as direct access

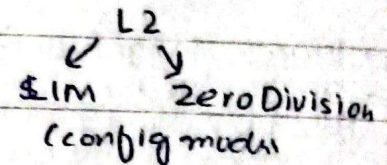
L2 Cont:

→ Reads return zero, and write are not persistently stored
 It is like a Placeholder region.

ECC:

→ Even Parity: Even number of ones.

→ Odd Parity: Odd numbers of ones.



We had $2^4 = 16$ combinations, out of which we selected 8 ~~even~~ parity codes. Because, let say we had give 0000 and 0001. 1 bit changed, yet both were valid so, error detection failed.

So, if parity becomes odd from even → issue detect
 But for this we added one bit "parity bit" that helps making invalid codes to no error. error in n bits.

→ $2^r \geq 1 + 1 \rightarrow$ error can $2^r = 1 + n$ point to exact bit or say no error

$2^r \geq 2^1 \rightarrow r=1$ 1 parity bit needed. if two valid codes bit errors you want to detect n bits apart. flipping k bit will

→ $d_{min} \geq k+1$ (Detection)

k represent the number of bits that will differ. result is invalid. We need all numbers valid will be 2 if $k=1$ make code invalid.

→ $d_{min} \geq 2k+1$ (Correction)
 to correct 1 bit error.

$2(1)+1 \Rightarrow 3$ is the min-distance

mean valid code words 000 then 111

So 1 bit error possibility 001 010 100 & 110 101 011

correct to 000

correct to 111

So, receiver can correct them

91r 52r

16

(Odd Parity → flags on 1 or when even

→ Parity bit & position (Even Parity → flags when odd 1s)

Suppose we send 7 bits

$D_4 D_3 D_2 D_1 D_0 P_2 P_1$

P_1 checks when $LSB = 1$: P_2 : 2, 3, 6, 7

Position	LSB=1
1	001
3	011
5	101
7	111

P_4 : 4, 5, 6, 7

let say: $D_1 = 1, D_2 = 0, D_3 = 1, D_4 = 1$

7 6 5 4 3 2 1
1 1 0 $P_4 P_2 P_1$

P_1 calculated: 101 $P_1 \rightarrow$ to keep ones even $P_1 = 0$

$P_2 \rightarrow$ 111 $P_2 \rightarrow P_2 = 1$ (so ones are 4)

$P_4 \rightarrow$ 110 $P_4 \rightarrow P_4 = 0$

so fixed: 1100110

Suppose bit 5 flips: 1110110

P_1 check: 1110 $\rightarrow$ odd parity: report error

→ Error CC: example 8-bit data

LEC II

Since, $n = 8$ bits

step 1: Choose r .

$$2^r \geq n + r + 1 \rightarrow \text{global parity}$$

$r = 4$

$$16 \geq 8 + 4 + 1$$

$$16 \geq 13$$

Hence, $r = 4 = \text{parity bits}$

+1 $\rightarrow$ global parity

Hence, 13 bits in total.

Parity bits are placed at power of two

P_1, P_2, P_4, P_8

remaining pos. contain data

step 2: fill data bits (include)

Q: Voltage 5V to 3.3V so power?

$$P \propto V^2$$

$$\frac{P_2}{P_1} = \left(\frac{V_2}{V_1}\right)^2 \Rightarrow \frac{P_2}{P_1} = \left(\frac{15}{13.3}\right)^2 \Rightarrow 0.4356$$

$$P_2 = 0.4356 P_1$$

$$\Delta P = P_2 - P_1$$

$$= 0.4356 P_1 - P_1$$

$$\Delta P = -56.44\%$$

$\rightarrow$ Reduction

DVFS example:

We are only given V_{dd} .

$$V_{dd} = 5.0V \quad E_1 = 40nJ$$

So if V_{dd}

Case I: $E_2 = 4$

$$\frac{E_{new}}{E_{old}} = \left(\frac{V_{new}}{V_{old}}\right)^2 \Rightarrow E_{new} = \left(\frac{4}{5}\right)^2 \times 40nJ$$

$$E_{new} = 25.6nJ$$

$f_1 \propto V_{dd} \Rightarrow 50MHz$ (given in text)

$$C_1 = 20pF \quad f_2 = 10 \times 4 = 40MHz$$

$$T_2 = \frac{1}{40 \times 10^6} = \frac{1}{f_2} = 25$$

Case II: 2.5V

$$E_{new} = \left(\frac{2.5}{5}\right)^2 \times 40nJ = 10nJ$$

$$f_{max} = 2.5 \times 10 \Rightarrow 25$$

$$C_3 = \frac{1}{f_3} = \frac{1}{25MHz} = 40ns$$

cycle $\downarrow$ time
Crt = execution

$$t_1 = 20ns \quad t_2 = 25ns \quad t_3 = 40ns$$

DVFS examples schedule:

50MHz for 20s:

By Table

Energy per cycle: 40nJ

$$Freq = 50MHz = 50 \times 10^6 \frac{\text{cycles}}{s}$$

$$50 \text{ cycles: } f \times t = \text{cycles}$$

$$50 \times 10^6 \times 20 = 10^9 \text{ cycles}$$

$$E = \text{Energy per cycle} \times \text{cycle}$$

$$= 40nJ \times 10^9$$

$$E = 40J$$

Schedule 2

50MHz for 15s 25MHz for 10s

By Table

$$E_1 = 40nJ \rightarrow 50MHz$$

$$E_2 = 10nJ \rightarrow 25MHz$$

for E_1 :

$$\text{cycles} = 50 \times 10^6 \times 15$$

$$= 0.75 \times 10^9$$

$$E = 40 \times 10^{-9} \times 0.75 \times 10^9$$

$$= 30J$$

For E_2 :

$$\text{cycles} = 25 \times 10^6 \times 10$$

$$= 2.5 \times 10^8$$

$$= 0.25 \times 10^9$$

$$E = 10 \times 10^{-9} \times 0.25 \times 10^9$$

$$E = 2.5J$$

$$\text{Sum: } 30 + 2.5$$

$$= 32.5J$$

Schedule 3:

40MHz for 25s

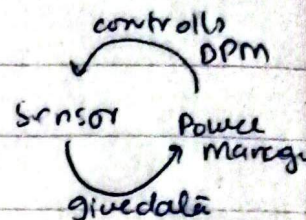
$$E = 25.6nJ$$

$$\text{cycle} = 40 \times 10^6 \times 25$$

$$= 10^9 \text{ cycle}$$

$$E = 25.6 \times 10^{-9} \times 10^9$$

$$E = 25.6J$$

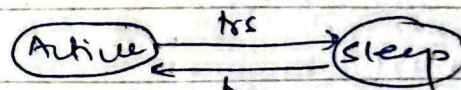


$\rightarrow$ We tried diff freq. at 25s.

LEC13

Power Down Cost:

$\rightarrow$ Over head incurred when a system or component transitions from active mode to low power sleep & state ban



includes trr & trs

$$\text{overhead} = trr + trs$$

this energy includes

- $\rightarrow$ Saving state
- $\rightarrow$ shutting down
- $\rightarrow$ Restoring clocks
- $\rightarrow$ Reloading cache & memory

$$\text{Overhead} = E_{tr} + E_{trr}$$

If idle time is very short, going to sleep may waste more energy than staying active hence, challenging decision

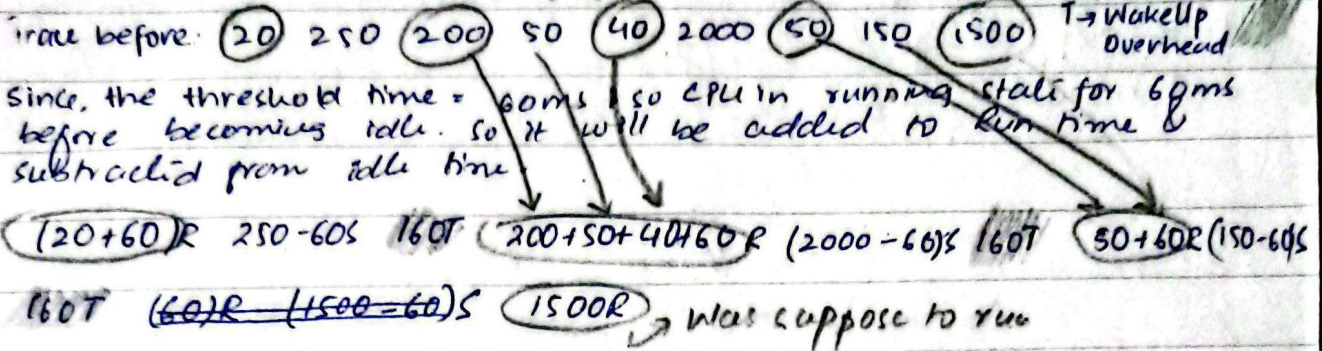
250,000Hz timer in s

$$1s \rightarrow 250,000 \text{ Hz}$$

LEC14

Slide 32:

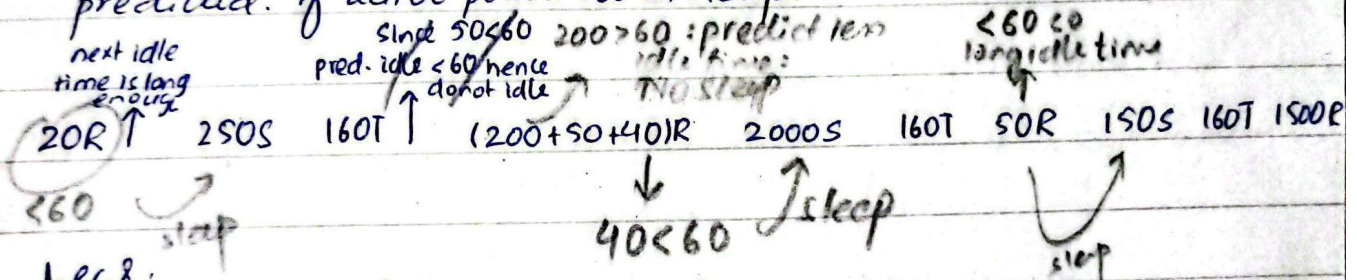
Fixed Time Out: Solution 1.



Slide 35:

Threshold Based Predictive Shutdown:

The processor sleeps immediately when long idle time predicted. if active period < 60 → sleep.



Lec 8:

$$\text{Total Cache Lines} = \frac{32 \text{ kiB}}{64 \text{ B (in each line)}} = \frac{32 \times 1024}{64} = 512 \text{ cache lines (in tot)}$$

$$\text{Number of Sets} = \frac{\text{Total lines}}{\text{Ways}} = \frac{512}{4} = 128 \text{ sets.} \quad \text{Each Way = One line}$$

Bytes in one way: $128 \times 64 = 8192 \text{ B.}$

	way[0]	way[1]	way[2]	way[3]
set 0	line	l	l	l
set 1	line	l	l	l
...	...	...	...	...
set 128	l	l	l	l

way[0] = entire direct col, one slot in every row, in a memory block is placed in a set.